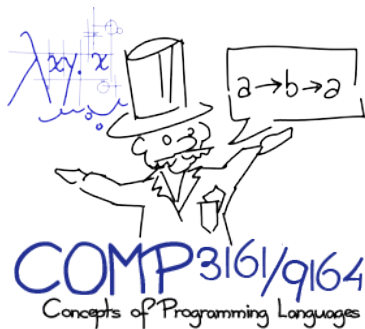




Hindley-Milner Type Inference

Craig McLaughlin
UNSW
Term 3 2023

Implicitly Typed MinHS

Explicitly typed languages are awkward to use¹. Ideally, we'd like the compiler to determine the types for us.

Example

What is the type of this function?

recfun $f\ x = \text{fst } x + 1$

We want the compiler to infer the **most general** type.

¹See Java

Implicitly Typed MinHS

Start with our polymorphic MinHS, then:

- **remove** type signatures from **recfun**, **let**, etc.
- **remove** explicit **type** abstractions, and type applications (the @ operator).
- **keep** $\forall$ -quantified types.

Implicitly Typed MinHS

Start with our polymorphic MinHS, then:

- **remove** type signatures from **recfun**, **let**, etc.
- **remove** explicit **type** abstractions, and type applications (the @ operator).
- **keep** $\forall$ -quantified types.
- **remove** recursive types, as we can't infer types for them.

see “whiteboard” for why.

Typing Rules

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{VAR}$$

Typing Rules

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{VAR}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 \ e_2 : \tau_2} \text{APP}$$

Typing Rules

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{VAR}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 \ e_2 : \tau_2} \text{APP}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (\text{Pair } e_1 \ e_2) : \tau_1 \times \tau_2} \text{CONJ}_I$$

Typing Rules

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{VAR}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \rightarrow \tau_2 \quad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1 \ e_2 : \tau_2} \text{APP}$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (\text{Pair } e_1 \ e_2) : \tau_1 \times \tau_2} \text{CONJ}_I$$

$$\frac{\Gamma \vdash e_1 : \text{Bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{If } e_1 \ e_2 \ e_3) : \tau} \text{I}_F$$

Primitive Operators

For convenience, we treat prim ops as **functions**, and place their types in the environment.

$$(+) : \text{Int} \rightarrow \text{Int} \rightarrow \text{Int}, \Gamma \vdash (\text{App} (\text{App} (+) (\text{Num } 2)) (\text{Num } 1)) : \text{Int}$$

Functions

$$\frac{x : \tau_1, f : \tau_1 \rightarrow \tau_2, \Gamma \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } (f.x. e)) : \tau_1 \rightarrow \tau_2} \text{FUNC}$$

Sum Types

$$\frac{\Gamma \vdash e : \tau_1}{\Gamma \vdash \text{InL } e : \tau_1 + \tau_2} \text{DISJ}_{I1}$$

$$\frac{\Gamma \vdash e : \tau_2}{\Gamma \vdash \text{InR } e : \tau_1 + \tau_2} \text{DISJ}_{I2}$$

Note that we allow the other side of the sum to be **any** type.

Polymorphism

If we have a polymorphic type, we can instantiate it to any type:

$$\frac{\Gamma \vdash e : \forall a. \tau}{\Gamma \vdash e : \tau[a := \rho]} \text{ALLE}$$

Polymorphism

If we have a polymorphic type, we can instantiate it to any type:

$$\frac{\Gamma \vdash e : \forall a. \tau}{\Gamma \vdash e : \tau[a := \rho]} \text{ALLE}$$

We can quantify over any variable that has not already been used.

$$\frac{\Gamma \vdash e : \tau \quad a \notin TV(\Gamma)}{\Gamma \vdash e : \forall a. \tau}_{\text{ALLI}}$$

(Where $TV(\Gamma)$ here is all type variables occurring free in the types of variables in Γ)

The Goal

We want **an algorithm** for type inference:

- With a clear **input** and **output**.
- Which **terminates**.
- Which is fully **deterministic**.

Typing Rules

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (\text{Pair } e_1 \ e_2) : \tau_1 \times \tau_2}$$

Can we use the existing typing rules as our algorithm?

`infer :: Context → Expr → Type`

Typing Rules

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (\text{Pair } e_1 \ e_2) : \tau_1 \times \tau_2}$$

Can we use the existing typing rules as our algorithm?

`infer :: Context → Expr → Type`

This approach can work for monomorphic types, but not polymorphic ones. Why not?

First Problem

$$\frac{\Gamma \vdash e : \forall a. \tau}{\Gamma \vdash e : \tau[a := \rho]} \text{ALLE}$$

The rule to add a $\forall$ -quantifier can **always** be applied:

$$\frac{\frac{\vdots}{\Gamma \vdash (\text{Num } 5) : \forall a. \forall b. \text{Int}}}{\Gamma \vdash (\text{Num } 5) : \forall a. \text{Int}} \text{ALLE}$$

$$\frac{\Gamma \vdash (\text{Num } 5) : \forall a. \text{Int}}{\Gamma \vdash (\text{Num } 5) : \text{Int}} \text{ALLE}$$

Read as an algorithm, the rules are **non-deterministic** – there are many possible rules for a given input. A depth-first search strategy may end up attempting infinite derivations.

Another Problem

$$\frac{\Gamma \vdash e : \forall a. \tau}{\Gamma \vdash e : \tau[a := \rho]} \text{ALLE}$$

The above rule can be applied at **any time** to a polymorphic type, even if it would break later typing derivations:

$$\frac{\Gamma \vdash \text{fst} : \forall a. \forall b. (a \times b) \rightarrow a \quad \dots}{\Gamma \vdash \text{fst} : (\text{Bool} \times \text{Bool}) \rightarrow \text{Bool} \quad \Gamma \vdash (\text{Pair } 1 \text{ True}) : (\text{Int} \times \text{Bool})} \Gamma \vdash (\text{Apply fst (Pair 1 True)}) : ???$$

Yet Another Problem

The rule for **recfun** mentions τ_2 in both input and output positions.

$$\frac{x : \tau_1, f : \tau_1 \rightarrow \tau_2, \Gamma \vdash e : \tau_2}{\Gamma \vdash (\text{Recfun } (f.x. e)) : \tau_1 \rightarrow \tau_2} \text{FUNC}$$

In order to infer τ_2 we must provide a context that includes τ_2 — this is circular. Any guess we make for τ_2 could be wrong.

Solution

We allow types to include *unknowns*, also known as *unification variables* or *schematic variables*. These are placeholders for types that we haven't worked out yet. We shall use α, β etc. for these.

Example

$(\text{Int} \times \alpha) \rightarrow \beta$ is the type of a function from tuples where the left side is `Int`, but no other details of the type have been determined yet.

As we encounter situations where two types should be equal, we *unify* the two types to determine what the unknown variables should be.

Example

See the lecture notes!

Back to Type Inference

We will **decompose** the typing judgement to allow for an additional output — an updated typing context which represents a *minimal information increase* over the input context in order to infer the type of the expression.

Inputs Expression, Context

Outputs Type, Context

We will write this as $\Gamma \vdash e \Rightarrow \tau \dashv \Gamma$, to make clear what are the inputs and outputs.

Generalisation

In our typing rules, we could generalise a type to a polymorphic type by introducing a $\forall$ at any point. We want to restrict this to only occur in a *syntax-directed* way.

Consider this example:

```
let  $f = (\text{recfun } f \ x = (x, x))$  in (fst ( $f$  4), fst ( $f$  True))
```

Where should generalisation happen?

Let-generalisation

To make type inference tractable, we will generalise only in **let** expressions.

Let-generalisation

To make type inference tractable, we will generalise only in **let** expressions.

This means that **let** expressions are now not just sugar for a function application. They actually play a vital role, as the place where generalisation happens.

Summary

- The rest of the rules are straightforward from their typing rules.
- We've specified a variant of algorithm $\mathcal{W}$ for type inference. Many other algorithms exist, for other kinds of type systems, including explicit *constraint-based* systems.
- This algorithm is restricted to the Hindley-Milner subset of decidable polymorphic instantiations, and requires that polymorphism is top-level — polymorphic functions are not first class.
- We still need an algorithm to compute the unifiers.